SENIOR ARCHITECTURAL SPECIFICATION & CODEBASE ANALYSIS

AGENTIC EXPENSE TRACKER

Complete Code Logic & Implementation Walkthrough

How a Senior Systems Architect Builds an AI-Powered Financial Engine Step-by-Step Without AI Shortcuts

---

Document Scope & Architectural Methodology
This engineering analysis reconstructs the Agentic Expense Tracker codebase in the strict chronological sequence a senior software engineer would follow when designing and building the system from scratch. It proceeds systematically through 17 phases: foundational infrastructure and cryptographic secrets, relational schema modeling, repository data-access isolation, service domain logic, thin controller endpoints, dual-engine AI agents with deterministic regex fallback, multi-channel gateways (Telegram, WhatsApp, Slack), asynchronous background task queues (ARQ + Redis), report visualizers, and a modern reactive TypeScript/React dashboard.

System Property	Architectural Specification / Implementation
Target Repository	cmdXSec/high/exp_tracker
Architectural Pattern	Modular Monolith • Clean Architecture • Repository Pattern
Backend Stack	Python 3.12, FastAPI, Asyncpg, SQLAlchemy 2.x ORM
Frontend Stack	React 19, TypeScript, Vite, Redux Toolkit, Tailwind
AI Agent Framework	Agno Multi-Provider (OpenAI, Claude, Gemini, Groq) + Regex Fallback
Channel Gateways	Telegram Bot API, Twilio WhatsApp, Slack Webhook, Web Chat
Async & Background Engine	Redis 7 + ARQ Task Queue (Async reports & cron subscriptions)
Security & Crypto	Argon2 / BCrypt, JWT Bearer, AES-Fernet at rest
Source Files Analyzed	65+ Production Files | ~5,500+ Lines of Code
Generated Date	October 06, 2026 at 23:09

Table of Contents & Build Sequence

The 17 sequential phases below follow the strict chronological progression a senior engineer executes when building a production-grade system. Jump directly to any section to review the architectural rationale, file implementations, and code logic.

Phase	Topic / Chapter	Focus Files & Scope	Page
1	Project Vision & Architecture Decisions	System design & pattern decisions	3
2	Infrastructure Foundation — Config, DB, Session	config.py, session.py, database engine	5
3	Security Layer — Passwords, JWT, Encryption	security/password.py, jwt.py, encryption.py	7
4	Domain Models — The Database Schema	models/user.py, expense.py, recurring.py, ...	8
5	Utility Belt — DateTime, Currency, Serialization	utils/datetime_utils.py, currency.py	10
6	Pydantic Schemas — API Contracts	schemas/user.py, expense.py, budget.py, ...	11
7	Repository Layer — Data Access	repositories/expense.py, user.py, base.py	12
8	Service Layer — Business Logic	services/expense.py, recurring.py, report.py	13
9	Controllers — Thin MVC Route Handlers	controllers/expense.py, auth.py, ...	15
10	AI Agent System — Agno ExpenseAgent	agents/expense_agent.py, context.py, tools/*.py	16
11	Channel Abstraction — Multi-Platform Messaging	channels/base.py, telegram.py, channel_factory.py	18
12	Report Engine — Data Aggregation & PNG Rendering	reports/engine.py, aggregators.py, matplotlib Agg	19
13	Background Workers — ARQ, Cron, Reminders	workers/tasks.py, cron.py, redis connection	20
14	Frontend Architecture — React + TypeScript + Vite	frontend/src/ (Redux, Tailwind, Lucide, Recharts)	21
15	Docker & Deployment — Composing Everything	docker-compose.yml, Dockerfiles, Nginx conf	22
16	Testing Strategy	tests/test_agent.py, test_expenses.py, conftest.py	23
17	Summary & Architectural Reflection	Key architectural takeaways & lessons	24

> ***WHY: The Sequential Rule:*** *Why start with Config and Crypto rather than the AI or UI? In mission-critical software, data boundaries, encryption, and transactional integrity dictate the system boundaries. Once the domain is modeled and secured, user interfaces and AI tools become straightforward consumers of the underlying architecture.*

Chapter 1: Project Vision & Architecture Decisions

1.1 — What Are We Building?

A multi-user expense management platform where users can record, query, edit, delete, and analyze expenses through a React web dashboard **and** through external messaging channels — WhatsApp (Twilio), Telegram (Bot API), and Slack (Webhooks). An embedded conversational AI agent understands natural language commands like *"I spent 450 on dinner"* and maps them to structured database operations.

1.2 — Why Modular Monolith?

A senior engineer choosing architecture for a **team of 1–5** and a product that needs to ship fast would pick a modular monolith over microservices. The reasoning:

- **Single deployable** — one container, one CI pipeline, one process to monitor.
- **Clear internal boundaries** — controllers, services, repositories, agents, channels are isolated modules. If you later need to extract a microservice (e.g., the AI agent), the interfaces are already clean.
- **No network overhead** — service-to-service calls are in-process Python function calls, not gRPC/REST hops.
- **Transactional consistency** — a single database transaction can atomically create an expense *and* record the conversation message. No distributed saga needed.

WHY: *Microservices are a scaling strategy, not an architecture goal. You earn the right to split later.*

1.3 — MVC + Service Layer + Repository Pattern

The backend follows a strict layered architecture. Each layer has one job:

Layer	Responsibility	Rule
Controller	Parse HTTP, validate auth, delegate, return response	ZERO business logic, ZERO SQL
Service	Orchestrate business rules, validation, multi-repo coordination	No HTTP concepts, no request/response objects
Repository	Execute database queries, return ORM entities	No business rules, only data access
Model	Define table schema, columns, relationships	Pure data definition, no behavior
Schema	Define API input/output contracts (Pydantic)	Validation only, no persistence logic

WHY: *This layering means if you replace PostgreSQL with DynamoDB tomorrow, only the Repository files change. If you swap FastAPI for Django, only the Controllers change. The business logic in Services remains untouched.*

1.4 — Core Design Principles

- **User Isolation** — Every query includes user_id. User A can never see User B's data.
- **Decimal Precision** — All money uses Decimal / Numeric(12,2). Never IEEE 754 floats.

• **Channel Inversion** — The core app depends on BaseChannel (abstract). Provider SDKs are decoupled adapters.

• **Zero Hardcoded Secrets** — All keys come from .env. Channel credentials are AES-Fernet encrypted at rest.

• **Offline-First AI** — If no LLM API key is configured, a deterministic regex-based intent parser handles all commands.

Chapter 2: Infrastructure Foundation — Config, Database, Session

A senior engineer starts here. Before writing a single API route, you need: environment configuration, a database connection, and a session management strategy. Everything else builds on top of these.

2.1 — Centralized Configuration (config.py)

■ backend/app/config.py

Uses pydantic-settings to load all configuration from .env files with type validation. The Settings class defines every configurable parameter: database URL, Redis URL, JWT secrets, CORS origins, encryption keys, and default LLM model.

```
class Settings(BaseSettings): model_config = SettingsConfigDict( env_file=('.env', '../.env'), env_file_encoding='utf-8', case_sensitive=False, extra='ignore' ) DATABASE_URL: str = Field(...) SECRET_KEY: str = Field(...) CREDENTIALS_ENCRYPTION_KEY: str = Field(...) # ... 15+ more fields settings = Settings() # singleton, imported everywhere
```

***WHY:** A single Settings singleton means every module imports from app.config import settings. No scattered os.getenv() calls. If a required key is missing, Pydantic fails at startup — not at runtime when a user hits an endpoint 3 hours later.*

The CORS_ORIGINS field has a custom @field_validator that accepts both comma-separated strings and Python lists, splitting them intelligently. The wildcard "*" is handled separately for development convenience.

2.2 — Database Engine & Base Model (database.py)

■ backend/app/database/database.py

Creates the async SQLAlchemy engine and session factory. Two key decisions here:

- **Async engine** — uses create_async_engine with asyncpg driver. This means the entire I/O path (HTTP request → DB query → HTTP response) is non-blocking. A single Python process can handle hundreds of concurrent requests without threads.
- **SafeVector type** — a custom TypeDecorator that uses pgvector on PostgreSQL but falls back to JSON on SQLite. This allows the test suite to run with in-memory SQLite while production uses pgvector for semantic search embeddings.

```
engine = create_async_engine( settings.DATABASE_URL, echo=settings.DB_ECHO, future=True, ) async_session_maker = async_sessionmaker( bind=engine, class_=AsyncSession, expire_on_commit=False, # ← critical for async )
```

***WHY:** expire_on_commit=False prevents SQLAlchemy from expiring all loaded attributes after commit. In async code, expired attributes would trigger lazy loads which are synchronous and would block the event loop. This is the #1 mistake developers make with async SQLAlchemy.*

2.3 — Session Dependency (session.py)

■ backend/app/database/session.py

The get_db() function is a FastAPI dependency generator that yields an async session. It follows the commit-on-success, rollback-on-error pattern:

```
async def get_db() -> AsyncGenerator[AsyncSession, None]: async with async_session_maker() as session: try: yield session await session.commit() except Exception: await session.rollback() raise finally: await session.close()
```

WHY: *This means controllers and services never call commit() or rollback() themselves. The session lifecycle is fully managed by the dependency. Services call flush() to push changes to the database within the transaction, and the dependency commits at the end if no exception was raised. This is the Unit of Work pattern.*

2.4 — Application Entrypoint (main.py)

■ backend/app/main.py

The FastAPI application is created with a lifespan context manager for startup/shutdown. On startup: tables are auto-created for SQLite (test mode), and the Telegram long-poller is started. On shutdown: the poller is stopped and the engine is disposed.

All 7 routers are registered under /api/v1 prefix. CORS middleware is configured dynamically based on settings.CORS_ORIGINS. A /health endpoint is exposed for Docker healthchecks.

Chapter 3: Security Layer — Passwords, JWT, Encryption

Security is not an afterthought — it's the second thing a senior engineer writes, right after the database connection. Three subsystems here: password hashing, JWT token lifecycle, and credential encryption.

3.1 — Password Hashing (password.py)

■ backend/app/security/password.py

Two pure functions: get_password_hash() and verify_password(). Uses bcrypt with auto-generated salt. The verify_password function catches all exceptions and returns False — it never leaks error information.

> ***WHY:*** *Bcrypt is deliberately slow (cost factor). It takes ~100ms to hash, which makes brute-force attacks infeasible. The salt is embedded in the hash string itself, so you don't need a separate salt column.*

3.2 — JWT Authentication (auth.py)

■ backend/app/security/auth.py

Three components:

- **create_access_token()** — builds a JWT with sub (user UUID), exp (expiration), and iat (issued-at). Signed with HS256 using SECRET_KEY.
- **decode_access_token()** — validates signature and expiration. Raises 401 Unauthorized with WWW-Authenticate: Bearer header on failure. Distinct error messages for expired vs. invalid tokens.
- **get_current_user()** — a FastAPI dependency that chains OAuth2PasswordBearer → decode_access_token → UserRepository.get_by_id(). Validates the user exists, parses the UUID from the sub claim, and checks is_active status. Returns a fully hydrated User ORM object.

> ***WHY:*** *The get_current_user dependency is injected into every protected route via Depends(). This means authentication logic is written once and enforced everywhere. A controller cannot accidentally skip auth.*

3.3 — Fernet Encryption (encryption.py)

■ backend/app/security/encryption.py

Channel credentials (Twilio API keys, Telegram bot tokens, Slack webhooks) are encrypted at rest using cryptography.fernet.Fernet — which is AES-128-CBC with SHA256 HMAC for integrity verification.

```
def encrypt_credentials(data: Dict) -> str: fernet = _get_fernet() raw = json.dumps(data)
return fernet.encrypt(raw.encode('utf-8')).decode('utf-8') def decrypt_credentials(token: str)
-> Dict: fernet = _get_fernet() decrypted = fernet.decrypt(token.encode('utf-8')) return
json.loads(decrypted.decode('utf-8'))
```

If no explicit CREDENTIALS_ENCRYPTION_KEY is set, a deterministic key is derived from SECRET_KEY using SHA-256 → base64. This is a pragmatic fallback for development environments.

> ***WHY:*** *Credentials are stored as Fernet tokens in the database. Even if the database is breached, the attacker cannot read Twilio/Telegram keys without the Fernet key. The API never returns decrypted credentials — only masked versions like sk-••••1234.*

Chapter 4: Domain Models — The Database Schema

With infrastructure and security in place, the next step is defining *what data exists*. 9 SQLAlchemy ORM models map to 9 PostgreSQL tables. Every model uses UUID primary keys, timezone-aware timestamps, and cascading deletes from the User root.

4.1 — User Model

■ backend/app/models/user.py

The User model is the root entity. All other models have a user_id foreign key pointing here with CASCADE delete. Key fields:

- id — UUID v4 primary key
- email — unique, indexed, used for login
- hashed_password — bcrypt hash (never the plaintext)
- currency / timezone — user preferences for formatting
- llm_provider, llm_model, llm_api_key_encrypted, llm_base_url — the user's personally configured AI provider, stored on the User row itself

7 relationship() declarations establish bidirectional ORM navigation. All use cascade='all, delete-orphan' so deleting a User cascades to all their expenses, conversations, channels, etc.

> ***WHY:*** *Storing LLM config on the User model (not in a separate table) is deliberate: it's a 1:1 relationship, and having it inline avoids an extra JOIN on every chat request. The encrypted API key column is String(512) because Fernet tokens are longer than the original plaintext.*

4.2 — Expense Model

■ backend/app/models/expense.py

The core financial entity. Uses Numeric(12, 2) for the amount column — this maps to PostgreSQL's exact-precision NUMERIC type, not a float.

- amount — Decimal with 12 digits, 2 decimal places
- category — denormalized string (fast reads) + optional category_id FK (for structured lookups)
- merchant — indexed for search queries
- occurred_at — when the expense happened (distinct from created_at)
- source — 'web', 'telegram', 'whatsapp', 'recurring' — tracks where the expense was created
- embedding — SafeVector(1536) for future semantic search via pgvector

4.3 — Supporting Models

Model	Table	Purpose
Category	categories	Predefined + custom expense categories with icon and color
RecurringExpense	recurring_expenses	Subscription rules (Netflix monthly, etc.) with frequency and next_occurrence
Conversation	conversations	Chat sessions per user per channel (web, telegram, whatsapp)

Message	messages	Individual messages within a conversation (user/assistant/system)
ChannelConnection	channel_connections	Encrypted channel credentials (Twilio SID, Telegram bot token)
Notification	notifications	Scheduled reminders with status tracking (pending/sent/failed)
Report	reports	Generated report metadata with JSON data and PNG file path

Chapter 5: Utility Belt — DateTime, Currency, Serialization

5.1 — DateTime Utilities (datetime.py)

■ backend/app/utils/datetime.py

Provides timezone-safe helper functions. Key functions:

- utc_now() — returns datetime.now(timezone.utc). Used as the default for every timestamp column.
- to_utc(dt) — ensures any datetime is timezone-aware UTC. Handles naive datetimes by assuming UTC.
- get_period_range(period) — converts named periods like 'today', 'this_month', 'previous_month', 'this_year' into (start_datetime, end_datetime) tuples.
- parse_month_string(period_str) — parses flexible month strings like '2026-09', 'Sep 2026', 'September' into date ranges. Handles both abbreviated and full month names.

WHY: *Centralizing date logic prevents 50 different places from computing 'this month'. The parse_month_string function is what allows the AI agent to handle 'Give me my September report' — it converts the natural language month into precise UTC bounds.*

5.2 — Currency Utilities (currency.py)

■ backend/app/utils/currency.py

A CURRENCY_SYMBOLS map and two functions: round_decimal() uses ROUND_HALF_UP (banker's rounding) to quantize values to 2 decimal places, and format_currency() adds the symbol prefix.

5.3 — Custom JSON Serialization (serialization.py)

■ backend/app/utils/serialization.py

A CustomJSONEncoder that handles UUID → string, Decimal → float, and datetime → ISO 8601 string. Used when serializing report data to JSON columns in PostgreSQL.

Chapter 6: Pydantic Schemas — API Contracts

Schemas define what goes in and out of the API. They are *not* the database models — they are the **contract** between the frontend and backend. Using Pydantic v2 with model_validate() and from_attributes = True for seamless ORM-to-schema conversion.

Module	Schemas	Purpose
auth.py	UserCreate, UserLogin, UserResponse, Token, TokenPayload	Registration/login input, user profile output, JWT payload
expense.py	ExpenseCreate, ExpenseUpdate, ExpenseResponse, ExpenseFilter, ExpenseListResponse, CategoryBreakdown	Expense CRUD with validation, filtering, pagination, and category listing
analytics.py	SpendingSummary, CategoryBreakdown, SpendingTrend, DayOfWeekPattern, TopMerchant, MonthlyComparison, AnalyticsDashboard	Computed/aggregated read-only responses for analytics endpoints
conversation.py	MessageCreate, MessageResponse, ConversationResponse, ChatRequest, ChatResponse	Chat I/O with conversation threading
channel.py	ChannelConnectionCreate/Update/Response, ChannelVerifyRequest/Response, IncomingWebhookPayload	Managing messaging channel connections and webhook normalization
report.py	ReportGenerateRequest, ReportResponse	Report trigger and metadata output
llm.py	LLMConfigUpdate, LLMConfigResponse, LLMFetchModelsRequest, LLMFetchModelsResponse	User LLM config with masked API key and curl command echoing

Key design decision: ExpenseCreate.amount has a @field_validator that automatically rounds to 2 decimal places using round_decimal(). This means the API enforces financial precision at the boundary — before the value ever reaches the service layer.

> ***WHY:*** *If you validate at the schema level, invalid data is rejected before it touches the database. If you validate at the service level, you've already allocated memory and started a transaction for garbage data. Always push validation as close to the boundary as possible.*

Chapter 7: Repository Layer — Data Access

7.1 — BaseRepository[T] — Generic CRUD

■ backend/app/repositories/base_repository.py

A generic repository using Python's Generic[T] to provide get(), create(), update(), delete(), and list() for any model. Uses flush() instead of commit() — the session dependency handles the final commit.

7.2 — ExpenseRepository — The Workhorse

■ backend/app/repositories/expense_repository.py

The most complex repository with 261 lines. Provides:

- get_by_user() — enforces WHERE id = ? AND user_id = ? for strict isolation
- list_by_user() — dynamic query building with optional filters (date range, category, merchant), pagination, count, and ORDER BY occurred_at DESC
- get_summary() — SUM(amount), COUNT(id), average calculation
- get_category_breakdown() — GROUP BY category with percentage calculation
- get_daily_breakdown() — GROUP BY date(occurred_at) for trend charts
- get_day_of_week_breakdown() — in-Python weekday aggregation (avoids SQLite/Postgres DOW dialect differences)
- get_top_merchants() — GROUP BY merchant ORDER BY total DESC LIMIT 5
- search_expenses() — LIKE search across merchant, description, and category

WHY: *The day-of-week aggregation is done in Python (not SQL) because SQLite's strftime('%w') and PostgreSQL's EXTRACT(DOW) return different numbering. Since the test suite uses SQLite, the repository does the grouping in Python for cross-dialect compatibility.*

7.3 — Other Repositories

- **UserRepository** — extends BaseRepository with get_by_email() for login lookup
- **CategoryRepository** — list_for_user() returns system defaults + user customs. Has seed_defaults() that auto-creates 10 standard categories on first access.
- **ConversationRepository** — get_or_create() for session management, add_message() to append messages and update the conversation timestamp
- **ChannelRepository** — get_by_external_id() for webhook identity resolution, get_user_connection() for credential retrieval
- **RecurringExpenseRepository** — get_due_expenses(up_to_date) for the background worker to find all subscriptions that need to be materialized

Chapter 8: Service Layer — Business Logic

Services orchestrate business rules. They call repositories, apply domain logic, and return results. They never touch HTTP concepts (Request, Response, status codes) — those belong to controllers.

8.1 — AuthService

■ backend/app/services/auth_service.py

register() checks for duplicate emails, hashes the password, creates the User, and seeds default categories. login() verifies credentials, checks is_active, and issues a JWT with the user UUID in sub.

8.2 — ExpenseService

■ backend/app/services/expense_service.py

Handles expense CRUD with category resolution. On create_expense(), it:

- 1. Looks up the category by name (case-insensitive) in the CategoryRepository
- 2. If found, uses the official category name and ID; if not, uses the user's input title-cased
- 3. Rounds the amount using round_decimal()
- 4. Normalizes the currency to uppercase, the source to lowercase
- 5. Converts occurred_at to UTC

WHY: *Category resolution is important because the user might type 'food' but the system category is 'Food & Dining'. The service handles this mapping so the agent and the web form produce consistent data.*

8.3 — AnalyticsService

■ backend/app/services/analytics_service.py

Computes the complete financial dashboard in a single method: get_dashboard_analytics(). It calls 5 repository methods in sequence (summary, categories, daily trends, day-of-week, top merchants) and optionally computes a month-over-month comparison with percentage change.

8.4 — ConversationService (The AI Orchestrator)

■ backend/app/services/conversation_service.py

This is the most critical service — it bridges the user's message to the AI agent. The handle_user_message() method:

- 1. Fetches the User to get currency, timezone, and **decrypted LLM credentials**
- 2. Gets or creates a Conversation session for the channel
- 3. Stores the incoming user message in the database
- 4. Assembles an AgentUserContext with all user details + LLM config
- 5. Creates an ExpenseAgent with the context and domain services
- 6. Calls agent.run(message) to get the AI response
- 7. Stores the assistant reply in the database
- 8. Returns the ChatResponse with the reply and conversation ID

8.5 — ChannelService

■ backend/app/services/channel_service.py

Manages encrypted channel credentials. connect_channel() optionally verifies credentials with the provider API before encrypting and storing. resolve_identity() maps an external sender ID (phone number, Telegram chat ID) to an internal user_id — this is how incoming webhook messages are associated with the correct user account.

8.6 — Other Services

- **RecurringExpenseService** — detects potential subscriptions by counting matching past expenses. process_due_expenses() materializes due recurring items into actual Expense records and advances the next occurrence date.
- **ReportService** — orchestrates report generation: compile data → render PNG → save file → persist Report record.
- **NotificationService** — CRUD for scheduled notifications with status tracking.
- **LLMFetcherService** — builds and executes curl commands against provider APIs to list available models. Supports OpenAI, Anthropic, Gemini, Groq, Ollama, and custom OpenAI-compatible endpoints.

Chapter 9: Controllers — Thin MVC Route Handlers

Controllers are **thin**. They parse the HTTP request, inject dependencies, call a service, and return the response. Zero business logic. Seven controllers, each following the same pattern:

```
@router.post('', response_model=ExpenseResponse, status_code=201) async def create_expense( data: ExpenseCreate, # ← Pydantic validates current_user: User = Depends(get_current_user), # ← JWT auth service: ExpenseService = Depends(get_expense_service) # ← DI ) -> ExpenseResponse: expense = await service.create_expense(current_user.id, data) return ExpenseResponse.model_validate(expense)
```

Controller	Prefix	Key Endpoints
AuthController	/auth	POST /register, POST /login, GET /me
ExpenseController	/expenses	POST, GET (list+filter), GET /{id}, PATCH /{id}, DELETE /{id}, GET /categories
AnalyticsController	/analytics	GET /dashboard?period=this_month
ConversationController	/conversations	POST /chat, GET (list), GET /{id}
ChannelController	/channels	POST /verify, POST /connect, GET /connections, PUT/DELETE /{id}, POST /webhook/{type}
ReportController	/reports	POST /generate, GET /{id}, GET /download/{id}
LLMController	/llm	GET /config, PUT /config, DELETE /config, POST /fetch-models

9.1 — The Webhook Controller (channel_controller.py)

The most complex controller. The POST /webhook/{channel_type} endpoint is **public** (no JWT) — it's called by Twilio/Telegram/Slack when a message arrives. It:

- 1. Parses the raw payload (JSON or form data)
- 2. Normalizes it via ChannelManager.parse_incoming()
- 3. Resolves the sender ID to an internal user via ChannelService.resolve_identity()
- 4. If unlinked, sends a 'please connect in Settings' fallback message
- 5. If linked, routes through ConversationService.handle_user_message()
- 6. Dispatches the AI reply back to the external channel

Chapter 10: AI Agent System — Agno ExpenseAgent

10.1 — Architecture

The agent system has 4 layers:

- **AgentUserContext** (context.py) — Pydantic model carrying user_id, currency, timezone, channel, and decrypted LLM credentials. Injected into every tool.
- **BaseAgent** (base_agent.py) — Abstract class defining build_agent(), get_tools(), run().
- **ExpenseAgent** (expense_agent.py) — The concrete agent with 302 lines. Dual execution path: Agno LLM agent (when configured) + deterministic intent parser (always available).
- **Tool Suites** (tools/) — 4 tool classes providing strongly-typed callable functions to the agent.

10.2 — The Dual Execution Engine

The run() method implements a graceful degradation chain:

- 1. If Agno agent is configured with a live LLM, call agent.arun(message)
- 2. If the LLM returns an error or is rate-limited, **fall back** to the deterministic parser
- 3. If no LLM is configured at all, use the deterministic parser directly

WHY: *This means the platform always works, even without an API key. The deterministic parser handles all core commands (create, delete, update, summarize, report). The LLM adds natural language flexibility on top.*

10.3 — Deterministic Intent Parser

The _parse_and_execute_intent() method uses regex pattern matching to classify user messages into 10 intent categories:

Priority	Intent	Pattern	Tool Called
0	Greeting/Help	/start, hi, hello, help	Returns help text
1	Report	'*report' with month name	report_tools.generate_monthly_report()
2	Today's Spending	'spend today', 'spent today'	analytics_tools.get_daily_summary()
3	Monthly Summary	'this month' + 'how much'	analytics_tools.get_monthly_summary()
4	Compare	'compare'	analytics_tools.compare_periods()
5	Category Breakdown	'category' + 'breakdown'	analytics_tools.get_category_breakdown()
6	Delete	'delete' or 'remove the'	expense_tools.search_and_delete_expense()
7	Update	'change' or 'update' + new amount	expense_tools.update_expense()
8	Subscription	'subscription', 'netflix', 'spotify'	recurring_tools.check_subscription_status()
9	List	'list' + 'expense'	expense_tools.list_expenses()
10	Record Expense	Amount + target ('450 on dinner')	expense_tools.create_expense()

The expense recording parser handles 4 input formats: *'I spent 450 on dinner'*, *'450 on dinner'*, *'dinner 450'*, and *'450 dinner'*. It uses a keyword-based auto-categorizer mapping words like 'uber', 'taxi' → Transportation, 'dinner', 'coffee' → Food & Dining.

10.4 — Tool Suites

- **ExpenseTools** — create, list, get, update, delete, search_and_delete. The create_expense tool also calls RecurringExpenseService.detect_potential_recurring() to suggest subscription tracking.
- **AnalyticsTools** — daily/monthly/yearly summaries, category breakdown, period comparison.
- **RecurringTools** — detect, create, list subscriptions, check status.
- **ReportTools** — generate monthly/yearly reports with PNG rendering.

10.5 — Multi-Provider LLM Support

The build_agent() method dynamically constructs the Agno Agent with the user's chosen provider. Supported: OpenAI, Anthropic (Claude), Google Gemini, Groq, Ollama (local), and custom OpenAI-compatible endpoints. Each provider imports its specific Agno model class and passes the user's API key.

Chapter 11: Channel Abstraction — Multi-Platform Messaging

11.1 — BaseChannel Contract

■ **backend/app/channels/base_channel.py**

An abstract base class defining 6 methods every channel must implement: send_message(), send_image(), parse_incoming(), validate_credentials(), connect(), disconnect(). Plus a ParsedIncomingMessage Pydantic model normalizing all incoming webhooks to {sender_id, text, metadata}.

> ***WHY:*** *This is the Strategy Pattern. The core application never imports httpx or Twilio SDKs directly. It only calls BaseChannel.send_message(). Adding a new channel (e.g., Discord) means creating one new file with 6 methods — zero changes to controllers, services, or agents.*

11.2 — Channel Implementations

- **WebChannel** — No-op adapter. Messages are delivered via REST responses. All methods return True.
- **TelegramChannel** — Uses httpx to call Telegram Bot API. Sends text via /sendMessage, photos via /sendPhoto. Validates with /getMe. Parses message.chat.id from webhook.
- **WhatsAppChannel** — Uses httpx against Twilio REST API. Handles whatsapp: prefix formatting. Validates with Twilio Account API. Parses From field stripping whatsapp: prefix.
- **SlackChannel** — Supports both Webhook URL (simple) and Bot Token (full API). Validates with /auth.test. Parses Slack Events API format.

11.3 — ChannelManager — The Dispatcher

■ **backend/app/channels/channel_manager.py**

Centralizes channel selection, credential decryption, and message dispatching. send_message() resolves the channel adapter, fetches encrypted credentials from the database, decrypts them with Fernet, and passes them to the adapter's send_message().

11.4 — Telegram Long-Poller

■ **backend/app/channels/telegram_poller.py**

A background asyncio.Task that polls Telegram's /getUpdates API for all active bot connections. This eliminates the need for a public URL (ngrok) during local development. Runs on application startup and processes each update through ConversationService.handle_user_message().

Chapter 12: Report Engine — Data Aggregation & PNG Rendering

12.1 — BaseReport → MonthlyReport → YearlyReport

Abstract BaseReport defines generate() (compile data) and render() (create PNG). MonthlyReport aggregates: summary KPIs, category breakdown, daily trends, top merchants, recurring subscriptions, and largest individual expenses. YearlyReport delegates to MonthlyReport with period='this_year'.

12.2 — Visual Renderer (renderer.py)

■ backend/app/reports/renderer.py

Uses matplotlib to create a dark-themed financial infographic card (12×8 inches at 150 DPI). Layout:

- Header: Period title + 'Agentic Expense Tracker' branding
- 3 Metric Cards: Total Spending (blue), Daily Average (green), Transaction Count (purple) — each in a rounded rectangle with accent-colored border
- Left Chart: Horizontal bar chart of top 7 spending categories with amount labels
- Right Chart: Area/line chart of daily spending trend with gradient fill
- Footer: 'Generated by Agentic Expense Platform' confidentiality note

The image is rendered to an in-memory BytesIO buffer, saved to disk, and stored in the Report record. The PNG can be downloaded via the API or sent directly to WhatsApp/Telegram.

Chapter 13: Background Workers — ARQ, Cron, Reminders

Uses ARQ (Async Redis Queue) for background task processing with Redis as the broker.

13.1 — Worker Configuration (worker.py)

■ backend/app/workers/worker.py

The WorkerSettings class registers 4 async functions and 2 cron jobs. Task functions: async_generate_report_job, send_notification_job, run_daily_reminder_job, run_recurring_expenses_job.

13.2 — Cron Schedule (scheduler.py)

- **00:05 UTC daily** — Process all due recurring expenses (materialize subscriptions into actual expense records)
- **20:00 UTC daily** — Daily spending reminder. Checks if each user has recorded expenses today; if not, sends a friendly nudge via their connected channel.

13.3 — Daily Reminder Logic (daily_reminder.py)

Iterates all active users, checks if they have channel connections, queries today's expense summary, and sends a personalized reminder if transaction_count == 0. Uses the user's first name for a friendly tone.

13.4 — Recurring Expense Processor (recurring_expenses.py)

Calls RecurringExpenseService.process_due_expenses() which finds all active recurring items where next_occurrence <= now, creates a materialized Expense record for each, and advances next_occurrence based on the frequency (daily/weekly/monthly/yearly).

Chapter 14: Frontend Architecture — React + TypeScript + Vite

14.1 — Technology Choices

- **React 18/19** — Component-based UI with hooks
- **TypeScript** — Full type safety with shared type definitions
- **Vite** — Lightning-fast HMR dev server and optimized production builds
- **Tailwind CSS v4** — Utility-first styling with modern light theme
- **TanStack Query** — Server state management with automatic caching, refetching, and staleness
- **Recharts** — Declarative chart components for financial visualizations

14.2 — Application Structure (App.tsx)

■ **frontend/src/App.tsx**

Simple tab-based SPA (no router needed). Manages auth state with useState. If not authenticated, shows Login/Register pages. If authenticated, shows the AppLayout with 4 tabs: Dashboard, Expenses, Reports, Settings.

14.3 — Pages

- **DashboardPage** — Fetches /analytics/dashboard and renders MetricCards, CategoryBreakdownChart, SpendingTrendChart, DayOfWeekChart, TopMerchantsList, and AIChatWidget.
- **ExpensesPage** — Table view with filters, pagination, and an ExpenseModal for creating/editing.
- **ReportsPage** — Period selector, generate button, and PNG download.
- **SettingsPage (64KB!)** — The largest component. Contains channel connection forms for WhatsApp, Telegram, Slack, and the full LLM provider configuration UI with curl command display and model dropdown.

14.4 — Components (12 files)

Each component is focused and reusable: MetricCards, CategoryBreakdownChart, DailySpendingChart, DayOfWeekChart, SpendingTrendChart, TopMerchantsList, RecentTransactionsTable, RecurringExpensesCard, ExpenseModal, AIChatWidget, Sidebar, Navbar.

14.5 — API Service Layer (services/)

6 service modules wrapping fetch calls: api.ts (base client with JWT header injection), auth.ts, expenses.ts, analytics.ts, chat.ts, reports.ts.

Chapter 15: Docker & Deployment — Composing Everything

■ docker-compose.yml

5 services orchestrated via Docker Compose:

Service	Image/Build	Purpose
postgres	pgvector/pgvector:pg16	PostgreSQL 16 with pgvector extension for semantic search
redis	redis:7-alpine	In-memory store for ARQ task queue and caching
backend	./backend Dockerfile	FastAPI application server (Uvicorn)
worker	./backend Dockerfile + ARQ cmd	Background task processor for cron jobs and async tasks
frontend	./frontend Dockerfile + nginx	React SPA served via Nginx, proxied to backend

Health checks ensure dependent services wait for readiness: PostgreSQL checks pg_isready, Redis checks redis-cli ping. The backend and worker depend on both being healthy. Volumes persist PostgreSQL and Redis data across container restarts.

The Makefile provides shortcuts: make dev (Docker compose), make test (pytest), make migrate (Alembic), make lint (Ruff), make format (Ruff format).

Chapter 16: Testing Strategy

The test suite uses pytest-asyncio with in-memory async SQLite. This means tests run without Docker, without PostgreSQL, and complete in seconds. The SafeVector type decorator falls back to JSON on SQLite, enabling pgvector columns to work in tests.

Test areas:

- **test_auth.py** — Registration, duplicate prevention, password validation, JWT tokens, unauthorized route blocking
- **test_expenses.py** — CRUD operations, Decimal precision, strict user data isolation (User B cannot access User A's data)
- **test_agent.py** — Conversational NLU: 'I spent 450 on dinner' → creates expense, 'What did I spend today?' → returns summary
- **test_channels.py** — BaseChannel contract adherence, webhook parsing, identity resolution
- **test_services.py** — Direct business logic unit tests and visual PNG report generation

Chapter 17: Summary & Architectural Reflection

The Senior Engineer's Build Sequence

Looking back at every chapter, the build order was deliberate and mirrors how a senior engineer would construct this system from scratch:

Step	What Was Built	Why This Order
1	config.py, database.py, session.py	Nothing works without config and a DB connection
2	password.py, auth.py, encryption.py	Security before features — never bolt it on later
3	All 9 models + __init__.py	Define the data shape before writing any logic
4	utils/ (datetime, currency, serialization)	Shared helpers used by everything above
5	All schemas (Pydantic contracts)	Define the API boundary before implementing it
6	All repositories (data access)	Encapsulate all SQL before business logic
7	All services (business logic)	Orchestrate repos with domain rules
8	All controllers (HTTP routes)	Thin wrappers connecting HTTP to services
9	Agent system (context, tools, agent)	AI layer on top of working services
10	Channel system (adapters, manager, poller)	Multi-platform messaging adapters
11	Report engine (data + renderer)	Visual output generation
12	Workers (ARQ, cron, reminders)	Asynchronous background processing
13	Frontend (React + TypeScript)	UI consuming the complete API
14	Docker Compose + Makefile	Package everything for deployment
15	Tests	Verify everything works in isolation

Key Architectural Takeaways

• **Layered architecture pays off** — Every module has one job. Swapping databases, LLM providers, or channels requires touching only one layer.

• **Security is infrastructure** — Password hashing, JWT, and encryption are written before the first API route.

• **Dual AI execution guarantees reliability** — The deterministic parser ensures 100% uptime regardless of LLM availability.

• **Channel abstraction enables scale** — Adding Discord or SMS support is a single file implementing 6 methods.

• **Financial precision is non-negotiable** — Decimal everywhere, ROUND_HALF_UP, Numeric(12,2) in PostgreSQL.

• **User isolation is enforced at every layer** — Repositories filter by user_id, services validate ownership, tools receive context-scoped user IDs.

• **Background workers decouple heavy operations** — Report rendering, recurring expense processing, and notifications run asynchronously.

End of Document

Total Source Files Analyzed: 65+ | Total Lines of Code: ~5,500+

Generated: October 06, 2026 at 23:09